Step 2 — Wrap It as a Single MCP Tool — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — The idea, keep it thin
- Hint 2 — The plan

Hint 1 — The idea, keep it thin

One `MCPServer`, one tool, a two-line body. The whole point is that wrapping is trivial — all the interesting design (trace, timeout) comes in Steps 3-4. Reuse Project 7's server shape: `from mcp.server.mcpserver import MCPServer`, `@mcp.tool()`, `mcp.run(transport="stdio")`. Build the config and model once at module level. Log to **stderr** only — stdout is the protocol channel.

Hint 2 — The plan

```
CONFIG = load_config(); MODEL = get_model(CONFIG)
mcp = MCPServer("deep-research-service")
@mcp.tool()
def deep_research(topic: str) -> str:
    """... docstring = what the caller sees ..."""
    result = run_pipeline(MODEL, topic, CONFIG.max_revision_rounds)
    return result.final_summary
mcp.run(transport="stdio")
```

Test with `mcp dev mcp_server.py` and call `deep_research` with the RAG topic -> the plain fact-checked summary, no trace yet. Resist adding a timeout or formatting here; keeping the body two lines is the point. The whole file set is in the Solution.